JavaScript Variables

What is a Variable?

A **variable** is a named container used to store data in memory. It allows you to save information that can be used, updated, or reused later in your program.

Variables can store different types of data, such as:

- Number
- String
- Boolean
- Null
- Undefined
- Object
- Array
- Function

Instead of writing the same value repeatedly, you can store it inside a variable and use it whenever needed.

Example

```
let name = "Rokon";
const age = 23;

console.log(name);
console.log(age);
```

Output

```
Rokon
```

```
23
```

Why Do We Use Variables?

Variables make programming much easier.

They help us to:

- Store data
- Reuse values
- Update values
- Make code clean and readable
- Reduce duplicate code

Without Variables

```
console.log("Rokon");
console.log("Rokon");
console.log("Rokon");
```

Output

```
Rokon
Rokon
Rokon
```

Explanation

The same value is written multiple times. If you want to change the name later, you must change every occurrence manually.

With Variables

```
let name = "Rokon";

console.log(name);
console.log(name);
console.log(name);
```

Output

```
Rokon
Rokon
Rokon
```

Explanation

The value is stored only once. If you change the variable, every place using that variable automatically gets the updated value.

Variable Naming Rules

A variable name must follow JavaScript rules.

- Must start with a letter, `_`, or `$`
- Cannot start with a number
- Cannot contain spaces
- Is case-sensitive
- Cannot use JavaScript reserved keywords

Valid Variable Names

```
let name = "Rokon";
let firstName = "Rokon";
let _age = 23;
let $price = 100;
```

Output

```
No Output
```

Explanation

These are valid variable names because they follow JavaScript naming rules.

Invalid Variable Names

```
let 1name = "Rokon";
let first name = "Rokon";
let let = 10;
```

Output

```
SyntaxError
```

Explanation

- `1name` starts with a number.
- `first name` contains a space.
- `let` is a reserved keyword.

JavaScript is Case-Sensitive

JavaScript treats uppercase and lowercase letters as different.

```
let name = "Rokon";
let Name = "Rahim";
let NAME = "Karim";

console.log(name);
console.log(Name);
console.log(NAME);
```

Output

```
Rokon
Rahim
Karim
```

Explanation

These are three different variables.

Variable Declaration

JavaScript provides three keywords to declare variables.

- var
- let
- const

Each behaves differently.

var

Definition

`var` is the old way of declaring variables.

Features:

- Function scoped
- Can be redeclared
- Can be reassigned

Syntax

```
var variableName = value;
```

Example

```
var name = "Rokon";

var name = "Rahim";

name = "Karim";

console.log(name);
```

Output

```
Karim
```

Explanation

- First declaration → Rokon

- Redeclared → Rahim
- Reassigned → Karim

Final value becomes **Karim**.

Scope Example

```
if (true) {
    var city = "Dhaka";
}

console.log(city);
```

Output

```
Dhaka
```

Explanation

Because `var` is function scoped, it ignores block scope.

let

Definition

`let` is the modern way of declaring variables.

Features:

- Block scoped
- Cannot be redeclared
- Can be reassigned

Syntax

```
let variableName = value;
```

Example

```
let age = 23;

age = 24;

console.log(age);
```

Output

```
24
```

Explanation

The variable value changes from **23** to **24**.

Redeclaration Error

```
let age = 23;

let age = 24;
```

Output

```
SyntaxError
```

Explanation

A `let` variable cannot be declared twice in the same block.

Scope Example

```
if (true) {
    let city = "Dhaka";
}

console.log(city);
```

Output

```
ReferenceError
```

Explanation

`city` exists only inside the block.

const

Definition

`const` creates a constant variable.

Features

- Block scoped

- Cannot be redeclared
- Cannot be reassigned
- Must be initialized

Syntax

```
const variableName = value;
```

Example

```
const PI = 3.1416;

console.log(PI);
```

Output

```
3.1416
```

Reassignment Error

```
const PI = 3.1416;

PI = 3.14;
```

Output

```
TypeError
```

Explanation

A constant variable cannot be assigned a new value.

const with Objects

```
const person = {
    name: "Rokon",
    age: 23
};

person.age = 24;

console.log(person);
```

Output

```
{
  name: 'Rokon',
  age: 24
}
```

Explanation

The object itself is constant, but its properties can still be modified.

const with Arrays

```
const numbers = [10, 20, 30];
```

```
numbers.push(40);

console.log(numbers);
```

Output

```
[10, 20, 30, 40]
```

Explanation

You cannot replace the array, but you can change its elements.

Primitive Data Types

Primitive data types store a single value.

```
let name = "Rokon";
let age = 23;
let isStudent = true;
let salary = null;
let address;

console.log(name);
console.log(age);
console.log(isStudent);
console.log(salary);
console.log(address);
```

Output

```
Rokon
23
true
```

```
null
undefined
```

Non-Primitive Data Types

Non-primitive values can store multiple pieces of information.

```
let person = {
    name: "Rokon",
    age: 23
};

let numbers = [10, 20, 30];

function greet() {
    console.log("Hello");
}

console.log(person);
console.log(numbers);
greet();
```

Output

```
{ name: 'Rokon', age: 23 }
[10,20,30]
Hello
```

Variable Scope

Scope determines where a variable can be accessed.

JavaScript has three scopes.

- Global Scope
- Function Scope

- Block Scope

Global Scope

```
let name = "Rokon";

function show() {
    console.log(name);
}

show();
```

Output

```
Rokon
```

Explanation

Global variables are accessible from anywhere.

Function Scope

```
function test() {
    var age = 23;

    console.log(age);
}

test();

console.log(age);
```

Output

```
23
ReferenceError
```

Explanation

The variable exists only inside the function.

Block Scope

```
if (true) {
    let city = "Dhaka";
    const country = "Bangladesh";
}

console.log(city);
console.log(country);
```

Output

```
ReferenceError
ReferenceError
```

Explanation

Both variables exist only inside the block.

Hoisting

Hoisting moves declarations to the top before execution.

var

```
console.log(name);

var name = "Rokon";
```

Output

```
undefined
```

Explanation

The declaration is hoisted, but the value is assigned later.

let

```
console.log(age);

let age = 23;
```

Output

```
ReferenceError
```

Explanation

`let` is hoisted but remains inside the Temporal Dead Zone until initialization.

const

```
console.log(PI);

const PI = 3.1416;
```

Output

```
ReferenceError
```

Explanation

`const` also stays inside the Temporal Dead Zone before initialization.

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone (TDZ)** is the period between entering a block and the line where a `let` or `const` variable is initialized.

```
{
    let age = 23;

    console.log(age);
}
```

Output

```
23
```

Explanation

The variable can only be used after its declaration.

Comparison Table

Feature	var	let	const
Scope	Function	Block	Block
Redeclare	Yes	No	No
Reassign	Yes	Yes	No
Hoisted	Yes	Yes (TDZ)	Yes (TDZ)
Must Initialize	No	No	Yes
Modern Use	Rare	Common	Most Preferred

Best Practices

Use `const` by default

```
const company = "Google";

console.log(company);
```

Output

```
Google
```

Use `let` when the value changes

```
let score = 0;

score++;
```

```
console.log(score);
```

Output

```
1
```

Avoid using `var`

Modern JavaScript projects rarely use `var` because it can introduce bugs due to function scope and redeclaration.

Summary

- Variables store data in memory.
- JavaScript has three variable keywords: `var`, `let`, and `const`.
- `const` is the safest and most preferred choice.
- Use `let` when the value needs to change.
- Avoid using `var` in modern JavaScript.
- Variable names are case-sensitive.
- Always choose meaningful variable names.
- Understanding scope and hoisting helps you avoid common JavaScript mistakes.

JavaScript Operators

What is an Operator?

An **operator** is a symbol that performs an operation on one or more values (operands).

Example

```
let sum = 10 + 5;
console.log(sum); // 15
```

Here:

- `10` and `5` are **operands**.
- `+` is the **operator**.

Types of Operators

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Increment & Decrement Operators
6. String Operators
7. Ternary Operator
8. Type Operators

1. Arithmetic Operators

Used to perform mathematical calculations.

Operator	Meaning	Example	Result
+	Addition	10 + 5	15
-	Subtraction	10 - 5	5
*	Multiplication	10 * 5	50
/	Division	10 / 5	2
%	Modulus (Remainder)	10 % 3	1
**	Exponent	2 ** 3	8

Example

```
let a = 10;
let b = 3;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
console.log(a ** b);
```

2. Assignment Operators

Used to assign values.

Operator	Example	Same As
=	x = 5	Assign value
+=	x += 2	x = x + 2
-=	x -= 2	x = x - 2
*=	x *= 2	x = x * 2
/=	x /= 2	x = x / 2
%=	x %= 2	x = x % 2

Example

```
let x = 10;

x += 5;
console.log(x); // 15

x -= 3;
console.log(x); // 12
```

3. Comparison Operators

Used to compare two values.

Operator	Meaning	Example
==	Equal (value only)	5 == "5" → true
===	Strict Equal	5 === "5" → false
!=	Not Equal	5 != 4
!==	Strict Not Equal	5 !== "5"
>	Greater Than	10 > 5
<	Less Than	5 < 10
>=	Greater Than or Equal	5 >= 5
<=	Less Than or Equal	5 <= 10

Example

```
console.log(10 > 5);
console.log(10 === "10");
console.log(10 == "10");
```

4. Logical Operators

Used to combine multiple conditions.

Operator	Meaning
&&	AND
||	OR
!	NOT

Example

```
let age = 20;
let hasID = true;

console.log(age >= 18 && hasID);
console.log(age < 18 || hasID);
console.log(!hasID);
```

5. Increment & Decrement Operators

Increase or decrease a value by 1.

Operator	Meaning
++	Increment
--	Decrement

Example

```
let count = 5;

count++;
console.log(count); // 6

count--;
console.log(count); // 5
```

6. String Operator

The `+` operator joins strings.

Example

```
let firstName = "MD";
let lastName = "Rokonuzzaman";
```

```
console.log(firstName + " " + lastName);
```

Output

```
MD Rokonuzzaman
```

7. Ternary Operator

A short way to write an if...else statement.

Syntax

```
condition ? trueValue : falseValue;
```

Example

```
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

8. Type Operator

The `typeof` operator returns the data type.

Example

```
console.log(typeof "Hello");
console.log(typeof 100);
console.log(typeof true);
console.log(typeof []);
console.log(typeof {});
```

Operator Priority (Precedence)

```
let result = 10 + 5 * 2;
console.log(result);
```

Output

```
20
```

Because multiplication (`*`) has higher precedence than addition (`+`).

Summary

Category	Purpose
Arithmetic	Mathematical calculations
Assignment	Assign values
Comparison	Compare values
Logical	Combine conditions
Increment/Decrement	Increase or decrease by 1
String	Join strings
Ternary	Short if...else
Type	Check data type

Easy Way to Remember

- ➕ ➖ ✖️ ➗ → **Arithmetic**
- = += -= → **Assignment**
- == === > < → **Comparison**
- && || ! → **Logical**
- ++ -- → **Increment / Decrement**
- + (String) → **Concatenation**
- ? : → **Ternary**
- typeof → **Type Checking**

Interview Questions

Q1. What is the difference between `==` and `===`?

- `==` compares only values (type conversion happens).
- `===` compares both value and data type.

```
console.log(5 == "5");   // true
console.log(5 === "5");  // false
```

Q2. What is the difference between `&&` and `||`?

- `&&` (AND): All conditions must be true.
- `||` (OR): At least one condition must be true.

```
console.log(true && false); // false
console.log(true || false); // true
```

JavaScript Data Types

What is a Data Type?

A **data type** tells JavaScript what kind of value a variable stores.

Example

```
let name = "Rokon"; // String
let age = 23;       // Number
```

Types of Data

JavaScript has **2 main types** of data.

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

1. Primitive Data Types

Definition

A **Primitive Data Type** stores a **single value**. It is **immutable** (cannot be changed directly).

There are **7 Primitive Data Types** in JavaScript.

Data Type	Description	Example
String	Text	`"Hello"`
Number	Integer or Decimal	`10` , `3.14`
Boolean	True or False	`true`
Undefined	Variable declared but no value assigned	`undefined`
Null	Intentionally empty value	`null`
BigInt	Very large integer	`1234567890123456789On`
Symbol	Unique identifier	`Symbol("id")`

Examples

String

```
let name = "Rokon";
console.log(name);
```

Number

```
let age = 23;
let price = 99.99;
```

Boolean

```
let isStudent = true;
```

Undefined

```
let city;
console.log(city);
```

Null

```
let car = null;
```

BigInt

```
let bigNumber = 1234567890123456789012345678890n;
```

Symbol

```
let id = Symbol("userId");
```

2. Non-Primitive (Reference) Data Types

Definition

A **Non-Primitive Data Type** can store **multiple values**.

These are called **Reference Types** because they are stored by reference (memory address).

Examples include:

- Object
- Array
- Function

Object

Objects store data in **key-value pairs**.

```
let student = {
    name: "Rokon",
    age: 23,
    country: "Bangladesh"
};

console.log(student.name);
```

Array

An array stores **multiple values** in a single variable.

```
let fruits = ["Apple", "Mango", "Orange"];

console.log(fruits[0]);
```

Function

A function is a reusable block of code.

```
function greet() {
    console.log("Hello!");
}

greet();
```

Primitive vs Non-Primitive

Feature	Primitive	Non-Primitive
Stores	Single Value	Multiple Values
Mutable	No (Immutable)	Yes (Mutable)

Feature	Primitive	Non-Primitive
Stored By	Value	Reference
Size	Fixed	Dynamic
Examples	String, Number, Boolean	Object, Array, Function

Easy Example

Primitive

```
let a = 10;
let b = a;

b = 20;

console.log(a); // 10
console.log(b); // 20
```

Explanation:

- `b` gets a **copy** of `a`.
- Changing `b` does **not** change `a`.

Non-Primitive

```
let person1 = {
    name: "Rokon"
};

let person2 = person1;

person2.name = "Rahim";

console.log(person1.name); // Rahim
console.log(person2.name); // Rahim
```

Explanation:

- `person1` and `person2` point to the **same object**.
- Changing one also changes the other.

Easy Way to Remember

Primitive = Single Box

```
age = 23
```

Each variable has its **own copy**.

Non-Primitive = 📍 Address

```
person ------\
              ---> { name: "Rokon" }
friend ------/
```

Both variables point to the **same object** in memory.

Interview Question

Q: What is the difference between Primitive and Non-Primitive Data Types?

Answer:

- Primitive data types store a **single value** and are copied by **value**.
- Non-primitive data types store **multiple values** and are copied by **reference**.
- Primitive values are immutable, while non-primitive values are mutable.